Web Systems Development

Authentication: your login, then everyone else's

Session 21 · Tue 11/10 · Part 1: logging in, from the pantry example · Part 2: sessions vs. tokens, "Log in with…", single sign-on

RPI

TODAY

Two parts: your login, then everyone else's

01	**Logging in**	Register, log in, stay logged in, log out — the real code from the pantry example, which is the shape of your project. Homework 9, which your team starts Friday.
02	**Who may do what**	Logged in is not the same as allowed. Auth middleware, a policy, and the field a user must never set.
03	**Sessions vs. tokens**	What if the client is not a browser? Bearer secrets, and why the session cookie is HttpOnly. One origin or two (CORS).
04	**"Log in with…" and single sign-on**	OAuth 2.0 and OpenID Connect in seven steps; how one registration with the school logs fifteen projects in.
05	**Friday: the login lab**	Add the login to your own api/ in class, with a check script. Homework 8 due; Homework 9 assigned.

Part 1 is 01 and 02 (about 50 minutes). Part 2 is 03 and 04 (about 40). Security, the OWASP Top 10 and six holes, was last week.

BUILDS ON

Where this sits: the login goes on top of your hardened api/

YOU ALREADY DID

HW 7

api/: a users table, a model, a seeder with three users, five routes

HW 8

the six defenses on it: headers, CSRF, bound queries, secrets, audit

week 11

the lab: an HttpOnly cookie is why XSS can't steal a login

midterm

rules.js: who may do what, decided in the page

TODAY

part 1

four routes and a cookie turn "user 1" into a real login

part 1

the rule from rules.js moves to the server: a policy

part 2

tokens for clients that are not browsers; OAuth; SSO

one idea

the request belongs to a stranger; decide on the server

NEXT

Fri 11/13

the login lab: add it to your api/, in class · HW 8 due

HW 9

team: the login and the write endpoints, with tests

Tue 11/17

cloud deployment: build, ship, run

week 13

one login for all fifteen projects: the platform hub

One running example: the pantry project (project/pantry-example-final), because it is the shape of your own project. Part 2 adds the course project platform, where your login goes in week 13.

Part 1 · Logging in

Homework 7 ended with a seeder that makes three users and routes that answer 501. Homework 9 turns that into a real login: register, log in, stay logged in, log out — and the write endpoints behind it.

Every line today is from project/pantry-example-final. Your api/ is the same shape.

What a login is, in four requests

1 · THE FORM

`POST /api/login`

The React login page sends an email and a password. Laravel checks the password against a hash it stored at registration.

web/src/pages/AuthPages.jsx → AuthController::login()

2 · THE COOKIE

`Set-Cookie: pantry_session=…; HttpOnly`

A good login starts a session. The browser keeps the cookie and sends it back, by itself, with every later request. JavaScript cannot read it.

bootstrap/app.php: statefulApi()

3 · EVERY LATER REQUEST

`GET /api/user → { "name": "Alex", … }`

auth:sanctum turns the cookie back into $request->user(). No cookie, or a stale one: 401, before your controller runs.

routes/api.php: the auth:sanctum group

4 · THE RULE

`DELETE /api/items/7 → 403`

Logged in is not the same as allowed. A policy says the owner may edit and only the RA may delete. The page hides buttons; the server decides.

app/Policies/ItemPolicy.php

Live: http://localhost:5181 in the example, log in as alex@example.edu, and watch these four in the Network tab. That is the whole of Homework 9's login.

Authentication is not authorization

AUTHENTICATION

Who are you?

Logging in: an email, a password, and from then on a session cookie that means "Alex". The wrong answer is 401 Unauthorized — "not logged in".

```
POST /api/items   as a guest   ->  401
```

AUTHORIZATION

Are you allowed to do this?

A rule about a logged-in user and a thing: the owner may edit an item; only the RA may delete one. The wrong answer is 403 Forbidden — "logged in, but no".

```
DELETE /api/items/7   as a resident   ->  403
```

Both are decided on the server, on every request. Your midterm's React pages had a rules.js that hid buttons. Hiding a button is good manners; it is not security. An attacker doesn't use your page — they send the request directly. The pantry final moved every rule into the API, and the page keeps its copy only to decide what to show.

Nine files. Three of them you already have.

File	What it does for the login	After Homework 7
`api/database/migrations/…_create_users_table.php`	password column; role with default 'user'	you have it (Laravel wrote it)
`api/app/Models/User.php`	'password' => 'hashed' cast; #[Fillable] allow-list without role	you have it — add the cast
`api/routes/api.php`	/register /login /logout /user; the auth:sanctum group around the writes	you have five routes — add four and the group
`api/app/Http/Controllers/Api/AuthController.php`	register(), login(), logout(), user()	new — about 60 lines
`api/app/Providers/AppServiceProvider.php`	RateLimiter::for('login') and 'register'	new — 5 lines
`api/bootstrap/app.php` · `api/.env`	$middleware->statefulApi() · SANCTUM_STATEFUL_DOMAINS	new — one line each
`api/app/Policies/ItemPolicy.php`	before(), create(), update(), delete(): who may do what	new — your rule from rules.js
`web/src/api.js` · `web/src/session.jsx`	login(), register(), logout(), currentUser(); who is logged in, for every page	new — copy from the example
`web/src/pages/AuthPages.jsx`	the two forms, with 422 errors next to the fields	new — the item form's pattern

Green: already in your api/ from Homework 7. Everything else is in project/pantry-example-final under the same path — read it there, then write yours.

Three ways to store a password. Which one is yours?

PREDICT Suppose the users table leaks (last week's drill 1 showed how). Which of these three lets the attacker log in as Alex tomorrow? Why is the middle one almost as bad as the first?

```
three versions of the users table, same password 'hunter2'
password = 'hunter2'                                        # plain text
password = sha256('hunter2') = f52fbd32b2b3b86ff88ef6c…     # a fast hash
password = bcrypt('hunter2') = $2y$12$Qn8bVZqW1r…          # bcrypt, 12 rounds, salted

api/app/Models/User.php      'password' => 'hashed',   // Hash::make on every save
api/.env                      BCRYPT_ROUNDS=12          // the work factor
```

SHA-256 is designed to be fast — billions of guesses a second on one GPU — and the same input always gives the same output, so common passwords are precomputed. bcrypt is slow on purpose (the work factor) and salted (two users with the same password get different hashes). The site can never tell you your password; it can only check a guess.

Register: a new row, and four lines that protect it

A07:2025 Authentication Failures *AuthController::register(), project/pantry-example-final*

```
public function register(Request $request) {
    $fields = $request->validate([
        'name'     => ['required', 'string', 'max:80'],
        'email'    => ['required', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'confirmed', Password::min(8)],
    ]);
    $user = User::create($fields)->fresh();  # 'password' => 'hashed'
    Auth::login($user);
    $request->session()->regenerate();
    return response()->json(['data' => $user], 201);
}
```

Green marks the lines that are there for security, not for the feature.

WHAT GOES WRONG

Three things go wrong at sign-up: two accounts with one email (which one logs in?); a mistyped password nobody can reproduce; a password stored as typed, so a leaked table is a list of working logins.

WHY EACH LINE WORKS

unique:users makes the database the judge of duplicates. confirmed requires a matching password_confirmation field. The hashed cast on User runs bcrypt on save, so the table never holds the password. Auth::login and regenerate() start a fresh session, exactly as login does.

THE FOUR, IN ONE LINE

validate · hashed cast · login · regenerate

Log in: four lines, each of which stops an attack

A07:2025 Authentication Failures *AuthController::login() and routes/api.php, pantry-example-final*

```
public function login(Request $request) {
    $credentials = $request->validate([
        'email' => ['required', 'email'], 'password' => ['required']]);
    if (! Auth::attempt($credentials)) {
        throw ValidationException::withMessages(
          ['email' => 'These credentials do not match our records.']);
    }
    $request->session()->regenerate();
    return ['data' => $request->user()];
}
routes/api.php  and  app/Providers/AppServiceProvider.php
Route::post('/login', [AuthController::class, 'login'])
      ->middleware('throttle:login');
RateLimiter::for('login', fn ($r) => Limit::perMinute(20)
    ->by(strtolower($r->input('email')).'|'.$r->ip()));
```

Green marks the lines that are there for security, not for the feature.

THE ATTACKS

Three attacks on any login form: guess passwords by the thousand; learn which emails have accounts from a helpful error ("wrong password" vs. "no such user"); plant a session id in the victim's browser before they log in, then share it.

WHY EACH LINE WORKS

Auth::attempt hashes the typed password and compares — the stored hash is never reversed. One message for both failures gives nothing away. regenerate() issues a new session id at login, so a planted one dies. throttle:login caps attempts at 20 a minute per email and address: 429 after that.

THE FOUR, IN ONE LINE

attempt · one message · regenerate · throttle

Two cookies: the session, and a token React carries by hand

After login() the browser holds a session cookie and sends it with every request on its own. Two lines of configuration make that happen for /api; then every write from React also carries a CSRF token, the way a Blade form carries @csrf.

```
api/bootstrap/app.php — requests from the React app get a session and CSRF protection
$middleware->statefulApi();
api/.env — the addresses the React app is served from (your web port, and Vite inside Docker)
SANCTUM_STATEFUL_DOMAINS=localhost:5181,web:5173

web/src/api.js — every write, before it is sent
await fetch('/sanctum/csrf-cookie', { credentials: 'include' });   // 1. ask Laravel for the token
const token = readCookie('XSRF-TOKEN');                            // 2. read it back
fetch('/api/items', { method: 'POST',
  credentials: 'include',                    // 3a. send the session cookie
  headers: { 'X-XSRF-TOKEN': token } });     // 3b. send the token back in a header
```

THE SESSION COOKIE

pantry_session · HttpOnly

The login itself. The browser attaches it to every request; JavaScript cannot read it, so a script injected into your page (last week, drill 2) cannot steal the login.

THE CSRF TOKEN

XSRF-TOKEN · readable on purpose

Not a secret: a value only pages from your own origin can read, echoed back in a header. It proves the request came from your page, not another site's (last week, drill 3).

React: ask who is logged in once, then post the form

```
web/src/session.jsx — who is logged in; every page calls useSession()
const [user, setUser] = useState(null);
const [status, setStatus] = useState('checking');  // until it answers
useEffect(() => {
  api.currentUser().then(setUser)
     .catch(() => setUser(null))          // 401: nobody is logged in
     .finally(() => setStatus('ready'));
}, []);
async function login(c) { setUser(await api.login(c)); }

web/src/pages/AuthPages.jsx — submit, the item form's pattern
try { await login(values); navigate('/'); }
catch (err) {
  setErrors(err.status === 422 ? err.errors   // one message per field
    : { form: [err.status === 429
        ? 'Too many tries. Wait a minute and try again.' : err.message] });
}
```

WHY A CONTEXT

The header, the shelf, the item page and the admin page all need to know who is logged in. One GET /api/user when the app loads, one answer handed to every component.

WHY "CHECKING"

Until the server answers, the app does not know. Without the third state, a page that needs a login flashes "please log in" at someone who is logged in.

WHY THE SERVER'S MESSAGES

validate() already wrote them: 422 carries one per field, 429 means throttled. The page shows them; it never decides them. (Week 9: lifting state, the same shape.)

Two more routes, and what 401 means

A07:2025 Authentication Failures *AuthController::logout() and user(), pantry-example-final*

```
routes/api.php
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user',    [AuthController::class, 'user']);
    Route::post('/logout', [AuthController::class, 'logout']);
    …the write routes…
});
AuthController.php
public function user(Request $request) { return ['data' => $request->user()]; }
public function logout(Request $request) {
    Auth::guard('web')->logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();
    return response()->noContent();              # 204
}
```

Green marks the lines that are there for security, not for the feature.

WHAT GOES WRONG

A session that never ends is a login that never ends: on a shared laptop, the next person is you. And a page that cannot ask "who am I?" has to guess, so it shows the wrong buttons.

WHY IT WORKS

Both routes sit inside the auth:sanctum group, so GET /api/user answers 401 when nobody is logged in — that 401 is what session.jsx turns into user = null. logout() forgets the user, throws the session away and issues a new CSRF token, so the old cookie is worth nothing.

THE TWO, IN ONE LINE

GET /user → who, or 401 · POST /logout → 204

Who may do what: the server decides, not the page

A01:2025 Broken Access Control *routes/api.php and app/Policies/ItemPolicy.php, pantry-example-final*

```
routes/api.php
Route::get('/items', [ItemController::class, 'index']);      # anyone
Route::middleware('auth:sanctum')->group(function () {      # or 401
    Route::post('/items', [ItemController::class, 'store']);
    Route::delete('/items/{item}', [ItemController::class, 'destroy']);
});
app/Policies/ItemPolicy.php  (found by name: Item -> ItemPolicy)
function before(User $u): ?bool { return $u->isAdmin() ? true : null; }
function update(User $u, Item $i): bool { return $i->user_id === $u->id; }
function delete(User $u, Item $i): bool { return false; }  # RA only
ItemController::destroy()
Gate::authorize('delete', $item);                            # or 403
```

Green marks the lines that decide who may do what.

THE ATTACK

The React page hides Delete from residents. An attacker doesn't use the page: curl -X DELETE /api/items/7 with their own cookie. If only the page checked, the shelf is empty by morning.

WHY IT WORKS

The middleware answers the first question before the controller runs: logged in? no → 401. The policy answers the second: owner or RA? no → 403, and Gate::authorize asks it on every write. before() lets the RA through everything.

THE FIX, IN ONE LINE

auth middleware → 401 · a policy → 403

A field the user must never set

A01:2025 Broken Access Control *User.php and ItemController::store(), pantry-example-final*

```
what an attacker sends to POST /api/register:
{ "name":"Mal", "email":"m@x.io", "password":"…", "role":"admin" }

app/Models/User.php
protected $guarded = [];              # unsafe: every column fillable
#[Fillable(['name', 'email', 'password'])]   # allow-list: no role
class User extends Authenticatable { … }

ItemController::store()
Item::create($request->all());              # user_id from the body
$request->user()->items()->create($request->validated());
```

Green is the fix. ~~Struck through~~ is what it replaces.

THE ATTACK

Register takes a JSON body, so the attacker adds "role": "admin". If the model fills any column a request names, they are the RA. Same shape: a user_id in the body makes a new item belong to someone else.

WHY THE FIX WORKS

Fillable is an allow-list of the columns a request may set. role is left off, so the extra field is silently ignored. The owner comes from the login — $request->user() — never from the body. The client proposes; the server decides.

THE FIX, IN ONE LINE

an allow-list of columns; owner from the login

Three requests sent with curl. Three status codes.

PREDICT Nobody clicked anything. Predict what the pantry API answers to each, and what role Mal ends up with.

```
1. no cookie at all
curl -X POST http://localhost:5181/api/items -d '{"name":"Milk"}'
2. Alex's cookie — a resident, not the RA — on someone else's item
curl -X DELETE http://localhost:5181/api/items/7 -b alex.cookies
3. a register body with one extra field
curl -X POST http://localhost:5181/api/register \
     -d '{"name":"Mal","email":"m@x.io","password":"hunter2hunter2","role":"admin"}'
```

Answers: 1 → 401, the auth:sanctum group, before the controller runs. 2 → 403, the policy: "Only the RA can remove items from the shelf." 3 → 201, and Mal is role "user" — the allow-list dropped the field. Three layers, three different lines of code, and none of them is in the React app.

SUMMARY · PART 1

Your users, in four lines

A07 · REGISTER

Register

validate (unique email, confirmed password), create with the hashed cast, Auth::login, regenerate. 201.

A07 · LOGIN

Log in

Auth::attempt against a bcrypt hash, one error message, regenerate the session, throttle. Not logged in → 401.

SESSION · COOKIES

Stay logged in

statefulApi() and SANCTUM_STATEFUL_DOMAINS; an HttpOnly session cookie, plus the XSRF token on every write. /user says who; /logout ends it.

A01 · POLICIES

Who may do what

auth:sanctum on the route (401), a policy for the rule (403), an allow-list of fillable columns. The owner comes from the login.

The idea under all of them: the request belongs to a stranger; the session and the policy belong to you. Decide everything on the server, and let the page only hide buttons.

What Homework 9 asks your team to build

1 **The login**

POST /register, /login, /logout and GET /user on your api/, as on the files slide. Seeded users can log in; a wrong password gets one message; twenty tries a minute gets 429.

2 **The write endpoints**

The three routes that answered 501 in Homework 7 now work, inside the auth:sanctum group, with validate() rules from your API.md. The owner comes from the login.

3 **One rule**

A policy for your resource: who may edit, who may delete. Both status codes — 401 for a guest, 403 for the wrong user — and the messages your API.md promised.

4 **The pages**

Log in, register and log out in your React app, with the server's 422 messages next to the fields. The buttons the midterm hid stay hidden; the server now enforces it.

The full page, with exactly what is graded, goes up Friday with the assignment. Everything on this slide is in project/pantry-example-final today. Later in the term (week 13) your login becomes a button on the shared project platform; nothing here changes for that.

Part 2 · Beyond your login

Your login is a password and a cookie. Three questions it does not answer: what if the client is not a browser? What if you would rather not hold passwords at all? And what if the school already knows who everyone is?

Sessions vs. tokens, "Log in with…", and single sign-on.

A session the browser holds, or a token the client holds

A SESSION — WHAT YOU BUILT

`Cookie: pantry_session=…`

The server keeps the state; the browser keeps a reference to it and sends it on its own, to its own site only. HttpOnly, so no script can read it. Log out = the server forgets. Right for a browser app that shares the API's address.

A TOKEN

`Authorization: Bearer 3|xK9…`

The client keeps the string and adds it to every request itself. Right for anything that is not your page: a phone app, a script, another server. The client must store it somewhere safe; a browser has nowhere an XSS bug cannot reach.

```
the same auth:sanctum middleware accepts either one: a session cookie, or a bearer token
issuing a token, after a normal login (e.g. POST /api/tokens):
$token = $request->user()->createToken('phone')->plainTextToken;   // shown once; stored hashed
using it, from anything that is not a browser:
curl http://localhost:5181/api/user -H "Authorization: Bearer 3|xK9…"
revoking every token this person has:
$request->user()->tokens()->delete();
```

The rule: a browser app on your own origin gets a session cookie; everything else gets a token. Your project needs only the first. Sanctum gives you both with the same middleware, which is why the route group did not change.

Remove the Vite proxy. Which request still works?

PREDICT The pantry example's React page and API share one address because Vite forwards /api. Suppose the page at localhost:5173 called the API at localhost:8000 directly instead:

```
fetch('http://localhost:8000/api/items')                                    // 1. read, no cookie
fetch('http://localhost:8000/api/items', { credentials: 'include' })        // 2. read, with the session cookie
fetch('http://localhost:8000/api/items', { method: 'POST', credentials: 'include',
     headers: { 'Content-Type': 'application/json' }, body: … })             // 3. write
```

Answers: 1 works — Laravel's default config/cors.php allows any origin to read /api/. 2 and 3 are blocked by the browser: a request that carries credentials needs the API to name the page's exact origin and say supports_credentials => true, and 3 is first sent as an OPTIONS "preflight" asking permission. Cookies are per site, so the session cookie would not even be the API's. The proxy makes all of it disappear: the browser sees one origin.*

Five strings. Which ones would you treat like a password?

PREDICT Each of these appears somewhere in a working app. For each, say what someone who copies it can do.

```
pantry_session=eyJpdiI6…          # the session cookie
XSRF-TOKEN=eyJpdiI6…              # the CSRF token (readable by your page on purpose)
Authorization: Bearer 3|xK9…       # a Sanctum API token
PANTRY_API_KEY=sk_test_000…        # an API key for another service, in .env
?code=def502…&state=…              # the one-time code in a "Log in with…" redirect
```

Answer: all but the CSRF token are bearer secrets — whoever holds the string is you, no password needed. So each gets the password treatment: never in a URL (history, logs, referrer), never in git, HttpOnly where a cookie can be, stored hashed where the server keeps it (Laravel hashes passwords and Sanctum hashes tokens), and short-lived (the OAuth code is good once, for a minute). The CSRF token is the odd one out: it proves which page sent a request, and is useless to anyone off your origin.

Four ways to answer "who are you?"

Pattern	Who checks the password	What your app stores	Use it when
Password + session (Part 1)	you	a bcrypt hash per user; a session per login	your own users, in a browser app on your origin — your project
Password + API token	you	the hash, plus hashed tokens with an expiry	the client is not your page: a phone app, a script, another server
"Log in with Google / GitHub" — OAuth 2.0 + OpenID Connect	they do	their stable id for the person, and your own session	you would rather not hold passwords, and your users already have that account
School single sign-on — SAML or OpenID Connect with the campus identity provider	the school	their id (and email, name, affiliation), and your own session	everyone belongs to one organization; multi-factor comes free; RPI uses Shibboleth
Passkeys / magic links	nobody — a device, or an inbox	a public key, or nothing	mentioned so you recognize them; not this course

The last column of every row ends the same way: "...and your own session." Whoever checks the password, your app still does Part 1's step 7: find the local user, log them in, regenerate. The patterns differ only in who answers the password form.

The round trip: seven steps, and the last is yours

1 **Redirect**

Your app sends the browser to the provider with your client_id, your redirect_uri, a random state, and a code_challenge (a hash of a random verifier it keeps).

2 **They log in there**

The person types their password on the provider's page. It never touches your app. The provider may ask "allow Pantry to see your name and email?"

3 **Back with a code**

The provider redirects to your redirect_uri with a one-time code and the same state. Your app checks the state against what it kept.

4 **Code for a token**

Server to server: your app posts the code, its client_secret and the verifier. The provider answers with an access token. No browser involved.

5 **Who is this?**

Your app asks the provider's userinfo endpoint with the token: a stable id (sub), a name, an email.

6 **Find or create**

Your users table: find by their id, else by email, else insert. Name and email come from them, never from the request body.

7 **Your own session**

Auth::login(); session()->regenerate(). Part 1, unchanged. From here on the provider is out of the picture until the next login.

OAuth 2.0 "authorization code" with PKCE, plus an OpenID Connect userinfo call. "Log in with Google", "Log in with GitHub" and the course platform all run exactly this.

Step 3 to 7 in Laravel: the callback route

A07:2025 Authentication Failures *HubAuthController::callback(), the course platform (Socialite)*

```
GET /auth/callback?code=…&state=…
public function callback(Request $request, HubUserSync $sync) {
    $remote = Socialite::driver('hub')->user();
        # checks state; trades code + verifier for a token; asks userinfo
    $user = $sync($remote);
        # find by their id, else by email, else create — forceFill, not the body
    Auth::guard('web')->login($user);          # Part 1, step 7
    $request->session()->regenerate();
    return redirect($return ?? '/');
}
routes: /auth/redirect (step 1), /auth/callback (this), /auth/logout
in your own project: composer require laravel/socialite, one provider per service
```

Green marks what the provider replaces: everything the password form used to do.

THE ATTACK

Someone sends your callback URL to a victim with THEIR code in it. If your app accepted it, the victim would be logged in as the attacker and type their secrets into the attacker's account. Or a stolen code is replayed from another app.

WHY IT WORKS

state is random, kept in your own cookie, and must match: a callback nobody started is refused (CSRF on the login itself). The code alone is useless: step 4 needs the client_secret and the PKCE verifier, which only your server has. And the provider only ever redirects to the redirect_uri you registered.

IN ONE LINE

Socialite::driver(…)->user(), then Part 1

Match each secret to the attack it stops

PREDICT Each one exists because of one specific attack. Pair them with a partner, then we check.

```
state              the random value your app keeps in a cookie and expects back in step 3
code_verifier      the random value whose hash went out in step 1 and which goes back in step 4
client_secret      the password your SERVER has for the provider, used only in step 4
redirect_uri       registered with the provider in advance; step 3 goes nowhere else
the code itself    one use, about a minute, exchanged in step 4 and never seen again
```

state: a callback nobody started — logging you in as someone else (CSRF on the login). code_verifier (PKCE): a code stolen in transit is useless without it. client_secret: another app pretending to be yours. redirect_uri: sending your code to the attacker's site. A short one-time code instead of the token in the URL: URLs leak — history, referrer headers, proxy logs — so what leaks must be worthless by the time anyone reads it.

Fifteen projects, one registration: single sign-on

THE SCHOOL'S IDENTITY PROVIDER

`RPI Shibboleth · SAML 2.0`

Knows every student. Registers each application by its exact callback URL, through a ticket to IT. Answers with a signed document: email, name, a stable id, affiliation. Never the password. Multi-factor is theirs.

▼

THE COURSE PLATFORM HUB

`one registered app · an OAuth provider`

The one application the school registers. Logs people in with the school (or a platform password on a laptop), keeps accounts and per-project roles, and is itself the provider the projects log in through — the seven steps, one level down.

▼

YOUR PROJECT

`three routes from a package`

/auth/redirect, /auth/callback, /auth/logout. Asks the hub "who is this, and what are they in my project?", mirrors the answer into its own users table, starts its own session. Part 1, step 7. Needs nothing from IT, ever.

The "single" in single sign-on: log in once on the hub, and every project's step 2 is instant — no password, no click. Log out on the hub, and they all end. Fifteen password tables become none. Where the project platform takes your login in week 13.

SUMMARY · PART 2

Beyond your login, in four lines

SESSIONS · TOKENS

Session vs. token

A browser app on your origin gets an HttpOnly cookie; anything else gets a bearer token. The same auth:sanctum middleware takes both.

SECRETS

Bearer secrets

Whoever holds the string is you: session ids, tokens, API keys, OAuth codes. Never in a URL or in git; hashed at rest; short-lived.

OAUTH · OIDC

"Log in with…"

OAuth 2.0 + OpenID Connect: redirect, they log in, a code comes back, your server trades it for a token, asks who, then Part 1 step 7.

SSO · SAML · WEEK 13

Single sign-on

One identity provider vouches for many apps. The school registers one app (our hub); the hub vouches for your project. Log in once.

The idea under all of them: whoever holds the secret is you. Hold as few as you can, for as short a time as you can — and when someone already knows the person, let them vouch.

Add the login to your api/, in class

Team work session, with the pantry example open beside you and a check script that sends the Part 1 requests to your API. Bring your api/ running — Homework 8 is due Friday, and the login goes on top of it.

0–15 MIN

Homework 8 check

Each team: docker compose watch up, /up answers, curl -i shows the four headers. We look at one or two api/ folders on the projector.

15–75 MIN

The nine files, in order

User model cast and allow-list · AuthController · the four routes and the auth:sanctum group · the rate limiter · statefulApi() and SANCTUM_STATEFUL_DOMAINS. Run the check after each step.

75–100 MIN

The pages

api.js login/register/logout/currentUser, session.jsx, the two forms. The midterm's rules.js stays; the server now enforces it.

```
inclass/wk12/check-login.sh — nine requests, PASS or FAIL each, against YOUR api/ (your web port)
bash inclass/wk12/check-login.sh http://localhost:8010
```

DUE & ASSIGNED

This Friday, 11/13

DUE FRIDAY

Homework 8 · team

Your hardened api/: the six defenses from the security lab on your own api/. Bring it running — Friday's lab builds on it.

Push whatever the lab produced to your team repo at the end of class.

ASSIGNED FRIDAY

Homework 9 · team

The login and the write endpoints, on your team's api/: register, log in, log out, auth:sanctum on the writes, validate() rules, one policy, and the pages. Part 1 today is the map; Friday's lab is the start; graded by using your app and curl. Due Fri 11/20.

NEXT WEEK · CLOUD DEPLOYMENT

Tue 11/17: build, ship, run — environments, secrets, migrations in production. Fri 11/20: the deployment lab, and Homework 9 due. Readings for Homework 9 (week 12 page): Laravel Validation, CSRF Protection, Session, Authentication and Authorization.

The request belongs to a stranger. The session belongs to you.

Check the password yourself, or let someone who already knows the person vouch for them — either way, the last step is yours: find the user, start a session, decide on the server what they may do. Friday, you build it.